Java OOPD Lab Assignment

L1- Encapsulation and polymorphism:

1.1 What is encapsulation?

Answer: Encapsulation is the OOPD principle of bundling data and the methods that operate on that data into a single unit, while locking your data inside a class and only allowing access through controlled methods. You mark field private so no one outside the class touch them directly. If they want to read or change the value, they must go through the public method, where you can add checks. Real-life-Analogy A medicine capsule. The medicine (data) is saled inside the shell. You can't touch the medicine directly —— you only interact with the capsule as a whole.

1.2 What is polymorphism?

Answer: polymorphism means "One name, many forms"— The same method name behaves differently depending on the situation.

- Compile-time (static) polymorphism: method overloading. The compiler decides which method to run based on the parameters, before the program even runs. Example: add(int a, int b) vs add(double a, double b).
- Run-time (dynamic) Polymorphism: method overriding. The compiler decides which method to run

while the program is actually Running, based on the real object type. Example: (Animal a = new dog(); a.Sound();— calls Dog's version even though a is declared as animal.

1.3: Write a Bankaccount class demonstrating encapsulation and polymorphism. Include a Main class calling both overloads.

Answer:

code:

```
class BankAccount{
    private double balance;
    public double getBalance(){
        return balace;
    }
public void deposit(double amount){
    balance += amount;
    System.out.println(" Deposited:"+amount);
 }
public void deposit(double amount,String remarks)
{ balance=+amount;
 System.out.println("Deposited: "+amount +"|Remark
s: " +remarks);
 }
}public class Main{
     public static void main(String[] args){
         BankAccount acc = new BankAccount();
         acc.deposit(500);
         acc.deposit(1000,"Salary credit");
```

```
        System.out.println("Balance : "+acc.getBalance());
    }
}
```

L2 – Method Overloading vs Overriding (Early vs Late Binding)

2.1: Compare Overloading vs Overriding: definition, class involved, parameters, return type, binding time.

Answer:

Point	Overloading	Overriding
Definition	Same method name, different parameters	Same method name and parameters, redefined in a sub class.
class involved	Same class	Parent and child class.
Parameters	Must be different	Must be the same
Return type	Can differ	Must be same or a subtype.
Binding time	Compile time (early binding)	Run time (late binding).

2.2: Explain early binding vs late binding. Why is Overriding resolved at run time and Overloading at compile time.

- Early Binding: The compiler already knows exactly which method to call, just by looking at the method signature. This happens with overloading static, private, and final method.

- Late Binding: The compiler doesn't know the final method to call in advance – The JVM checks the actual object at run time and calls right version. This happens with overriding. Overriding needs late binding because the same reference type can point to different object. Java can't know which one until the program is actually running.

2.3: Shape with area(). Overriding by circle and Rectangular. Add overloaded describe (String) / describe (String, int) in Spape (early binding). Show sample output.

Answer:

```
class Shape{
    public double area(){
        return 0;
    }
    public void describe (String name){
        System.out.println("Shape: "+name);
    }
    public void describe (String name, int sides)
    {System.out.println("Shape: "+name+", Sides:
        "+sides);
    }
}
```

```
class Circle extends Shape{
    private double radius;
    public Circle (double radius) {this radius=radius}
    public double area()
   {return Math.PI*radius*radius}
 }
class Rectangle extends Shape{
      private double length, width:
      public Rectangular (double length, double width)
{ this.length = length; this.width = width;
  }
public double area() {return length* width;}
}
public class Main{
   public static void Main (String[ ] args){
        Shape S1 = new Circle (5);
        Shape S2 = new Rectangle (4,6);
      System.out.println ("Circle Area : "+ S1 Area());
      System.out.println (" Rectangle Area : " +S2.area
       ());
       S1.describe ("Circle ");
       S2.describe (" Rectangle ", 4);
     }
  }
```

Sample Output:

```
Circle Area : 78.539816
Rectangle Area: 24.0
```

Shape: Circle
Shape : Rectangle , Side = 4

L3. Abstract class vs Interface:

3.1: Define Abstract class and Interface. List ≥ 4 structural differences.

Answer:

Abstract class: A class that cannot be instantiable directly. It can mix finished methods with unfinished methods that subclass must complete.

Interface: A contract listing method signature with no body. Any class that implements it must fill in those methods.

Aspect	Abstract class	Interface
Fields	Can have normal instance variebles	only public static final constant.
Constructor	Yes, Can have one	No.
Method bodies	Mix of implemented + abstract Method	Only default / static methods have bodies.
Multiple inheritance	A class can extend only one abstract class.	A class can implement Many interface.

3.2: When should we choose abstract class vs an interface? Give two real world scenarios.

Answer:

- Use an abstract class for an "is a" relationship where subclasses share common code. Example: Vehicle is base for Car, bike – they all share StartEngin() logic.
- Use an interface for a "can do" capability that unrelated classes might share. Example: Insurable — a car, a House, or even a person could all be insurable, even though they're not related by inheritance.

3.3: Vehicle (abstract, StartEngine()) implemented + abstract fuelType()) and insurable (interface, CalculatedPremium()). Car extends vehicle and implements Insurable. Comment on why each construct was chosen.

Code:

```
abstract class Vehicle{
    public void StartEngin(){
        System.out.println("Engin Started....");
    }
    public abstract String fuelType();
}
interface insurable{
        double calculatePremium();
}
class car extends vehicles implements insurable{
```

```
    public String fuelType(){
        return "Petrol";
    }
    public double calculatePremium(){
        return 15000.0
    }
}
public class Main{
    public static void main(String[] args){
        Car car = new car();
        car.StartEngin();
        System.out.println("Fuel: " + car.fuelType());
        System.out.println("Premium: " + car.calcu-
latepremium());
    }
}
```

Why abstract class for vehicle: it's clearly an is a relationship and startEngin() is shared code we don't want to rewrite in every subclass.
Why interface for Insurable: being insurable is a capability, not a type of vehicle — other unrelated classes could implement it too.

L4. Collection Framework

4.1: Compare Arraylist, vector, Linkedlist: Undelyting structure, Synchronization, insert/delete vs random access performance.

Answer:

Feather	Arraylist	Vector	Linkedlist
Structure	Resizable Array	Resizable Array	Doubly linked list.
Synchronized (thread-safe)	No	Yes	No
Random Access (get by index)	Fast	Fast	Slow
Insert/delete (middle)	Slow	Slow	Fast

4.2: Explain Set and its implementation Hashset, LinkedHashSet, Treeset. How does Treeset maintain Order?

Answer: Sets and its implementation: A set is a collection that does not allow duplicated elements.

- Hashset: no guaranteed order, Fastest.
- LinkedHashSet: keeps insertion order.
- Treeset: Keeps elements sorted automatically.

Treeset maintains order using a Red-Black Tree internally — Every time you add an animal,

its placed in the connect sorted position.

4.3: Stores 5 student names in an Arraylist and in a Treeset. Comment of ordering difference.

Answer:

Code:

```
import java.util.Arraylist;
import java.util.Treeset;

public class Main{
    public static void main (String[] args){
        Arraylist<String> List = new Arraylist<>();
        List.add("David");
        List.add("Alice");
        List.add("Ethan");
        list.add("Bob");
        List.add("Carol");

        System.out.println("Arraylist (insertion
        order):");
        for (String name: List){
            System.out.println(name);
        }
    Treeset<String> set = new Treeset<>(list);
    System.out.println("\n Treeset (sorted order):");
    for (String name: set){
        System.out.println(name);
    }
  }
}
```

Comment: The Arraylist prints names in the exact order they were added. The Treeset automatically sorts them alphabetically and would also remove any duplicates.

L5 – Multithreading And Custom Execption Handling:

5.1: List ≥3 ways to implement multithreading in Java. Which is preffered and why?

Answer:

Ways to implement multithreading:

1. Extending Threads and overriding run()
2. Implementing Runnable and passing it to a Thread object.
3. Using Executor Service / Callable – a thread pool manages threads for you and callable can return a result.

preffered: Runnable (or Executor Service) is generally preffered, because Java only allows single inheritance – if your class extends Thread, it can't extend anything else. Runnable keeps your class free to extend other classes, and separates the task from the thread that runs it.

5.2: Practical: two threads printing 1-5

Code:

```
class MyThread Extends Thread{
    public void run(){
        for(int i=1; i<=5;i++){
            System.out.println(" Thread (extends):"
+i );
 }
}
}
class My_Runnable implements Runnable{
    public void run(){
        for (int i = 1; i<=5; i++){
            System.out.println(" Thread (runnable):
"+i);
    }
  }
}
public class main{
    public static void main (String[] args){
        MyThread t1 = new MyThread ();
        Thread t2 = new Thread (new MyRunnable
());
    t1.start ();
    t2 .start ();
    }
 }
```

5.3 — Practical : Custom checked exception:

Code:

```
class InvalidRadiusException extends Exception{
    public InvalidRadiusException (String msg){
        super(msg);
    }
}
class circle{
    private double radius;
    public Circle(double radius) throws InvalidRadi-
ousException{
        if(radius<0){
            throw new InvalidRadiusException ("Radiu
s cannot be negative : "+ radius);
        }
        this.radius = radius;
    }
    public double area(){
        return Math.PI * radius * radius;
    }
}
import java.util.scanner;
public class Main{
    public static void Main (String[] args){
        Scanner sc = new Scanner (System.in);
        System.out.print ("Enter radius : ");
        double r = sc.nextDouble ();
        try{
            Circle c = new Circle (r);
```

```
        System.out.println("Area : " + c.area());
    } catch (InvalidRadiusException e) {
        System.out.println("Error : " + e.getMessa
ge());
    }
  }
}
```

L6 — JDBC with MySQL/Oracle (MVC Pattern)

6.1: Steps to connect Java to a database via JDBC.

1. Load / register the JDBC driver.
2. Open a connection: `Connection con = DriverManager.getConnection(url, user, password)`
3. Create a statement: `PreparedStatement ps = con.prepareStatement(SQL)`.
4. Run the update: `ps.executeQuery()`
5. Process the result using ResultSet (for SELECT queries)
6. close the connection.

6.2:— MVC pattern in a JDBC app:

MVC splits an app into three parts:-

Model:— The plain data class representing one records.

View — whatever collects input and shows output to the user.

Controller — The class handles logic and talks to the database, sitting between Model and view.

6.3 — practical: MVC - style Student insert

Code:

```
import java.sql.*;
import java.util.*;

class Student {
    private int id;
    private String name;
    private double cgpa;
    public Student(int id, String name, double cgpa
) {
        this.id = id;
        this.name = name;
        this.cgpa = cgpa;
    }
    public int getId() { return id; }
    public String getName() { return name; }
    public double getCgpa() { return cgpa; }
}
class StudentDAO {
    public void insert(Student s) throws SQLExe-
ption {
        String url = "jdbc:mysql://localhost:3306/
Student_db";
        try (Connection con = DriverManager.get
```

```
•getConnection(url, "root", password");
            PreparedStatement ps = con.prepareSta-
tement("INSERT INTO Students (id, name, cgpa)
VALUES(?, ?, ?)")){
        ps.setInt(1, s.getId());
        ps.setString(2, s.getname);
        ps.setDouble(3, s.getCgpa);
        ps.excecuteUpdate();
    }
  }
}
public class Main{
    public static void main(String[] args)throws SQL
Exception{
        Scanner sc = new Scanner (System.in);
        System.out.print("ID:");
        int id = sc.nextInt();
        System.out.print("Name : ");
        String name = sc.next();
        System.out.print("CGPA:");
        double cgpa = sc.nextDouble();
        Student s = new Student(id, name, cgpa);
        new StudentDAO().insert(s);
        System.out.println("Student Saved!");
    }
  }
```

L7 – JavaFX – House Loan Calculator

7.1: — JavaFx application Structure:

Application: — The base class every JavaFX app extends; JavaFX calls its start() methode to launch the GUI.

Stage — The actual windows on screen.

Scene — The content that fills the windows.

GridPane/VBox — layout containers that arrange buttons, labels and text fields.

Role of start( ): It's the entry point JavaFX calls automatically after lauch – this is where you build the scene, set up the layout, add event handlers, and show the stage.

7.2: — practical: GridPane layout

Code:
```
import javafx.application.Application;
import javafx.geometric.Insets;
import javafx.Sene.Scene;
import javafx.scene.control;
import javafx.scene.layout.GridPane;
import javafx.stage.Stage;

public class loanCalculator extends Application{
    TextField loanField = new TextField();
    TextField rateField = new TextField();
    TextField yearsField = new TextField();
    label resultLabel = new Label();
```

```
public void start(Stage stage){
    GridPane grid = new GridPane();
    grid.setPadding(new Insect(10));
    grid.setHgap(10);
    grid.setVgap(10);
}
private void calculate(){
      double P = Double.parseDouble(loanField.getText)
      double r = annualRate/100/12;
      double M = P*r* Math.pow(1+r,n)/(Math.pow
(1+r,n)-1);
     double T = M*n
     double D = T-P
   resultlabel.setText(String.format("Monthly:
%.2f|Total: %.2f| Different: %.2f ", M,T,D);
  }
  public static void main(String[]args]{
       launch(args);
  }
}
```

7.3 — Event Handler (already included above):

The calcBtn.setOnAction(e->calculate()) line runs the calculate() method when the button is clicked. It uses the standerd loan amortization formula:

r = AnnualRate/12/100

M = Pxrx(1+r)^n/[(1+r)^n-1]

T = M x n
D = T - P
Then it updates resultlabel with the monthly installment, Total payment and different.

L8 - Socket Programming and Java RMI (Chat System)

8.1 — Socket programming vs Java RMI - Sockets

Shocket — low-level, you send and receive raw text/bytes yourself over a Socket/ServerSocket connection. You control the exact format.

RMI (Remote Method Invocation): High-level, you call a method on a remote object almost like its local.

When to prefer each: Use sockets when you need full control over the protocol, cross-language communication, or lightweight custom messaging. Using RMI when you're working purely in Java and want to call remote methods directly without manually building a massage protocal.

8.2 — Server Code:

```
import java.io.*;
import java.net.*;
public class ChatServer{
    public static void main (String[] args) throws
IOException{
        ServerSocket serverSocket= new Server
```

```
        System.println ("Server waiting for connection");
        Socket.Socket = ServerSocket.accept();
        System.out.println ("Client connected!");
       String message = in.readline();
       System.out.println ("Client says: "+message);
       System.out.println ("Server received: " +
        message);
Socket.close();
ServerSocket.close();
    }
}
```

8.3 — practical Client code:

```
    import java.io.*;
    import java.io.*;
public class ChatClient {
   public static void main (String[] args) throws
IOException{
       Socket socket = new Scanner ("local host",
500);

    System.out.println ("Hello Server!");
    String reply = in.readline ();
    System.out.println ("Server reply:"+reply);
    Socket.close();
  }
}
```

L9 – Servlet + JSP + JDBC CRUD (Student record)

9.1: Steps to set up the project:

1. Create mysql database students_db and a student table with columns id, name, cgpa.
2. Set a Dynamic web project in your IDE add the MySQL connector.
3. Map the servlet to a URL patterns.
4. Build the JDBC connection string
5. Write the servlet, a DAO class and a JSP page.

9.2: —Practical: Servlet do post()

```
import.java.io.*;
import.java.sql.*;
import.jakarta.servlet*;
import.jakarta.servlet*;
public class AddStudentServlet extends HttpServlet{
      throws ServeletException, IOException{
 int id = Integer.parseInt(request.getParameter
 (id));
 String name = request.get parameter ("name");
double cgpa = Double.parseDouble(request.getpara-
meter ("cgpa"));
      ps.setInt(1,id);
      ps.SetString(2,name);
      ps.Set Double(3,cgpa);
      ps.executableUpdate();
      response.getwrite().println("successfully");
```

```
}catch(SQLException e){
    response.getWriter().println("Error:"+e.getMessage());
      }
    }
}
```

9.3:——Practical: Jsp page

code:

```
<%@page import="java.sql.*"%>
<html>
<body>
<h2>Student list</h2>
<table border="1">
<tr><th>ID</th><th>Name</th><th>CGPA</th></tr>
<%
    Connection con=DriverManager.
    Statement st=con.createStatement();
    while(rs.next()){
 %>
 <tr>
     <td><%=rs.getInt("Id")%></td>
     <td><%=rs.getString("name")%></td>
     <td><%=rs.getdouble("cgpa")%></td>
 </tr>
 <%
   }
 con.close();
%>
 </table>
 </body></html>
```